Quantitative Methods Audit

PORTFOLIO ALLOCATION — RISK PARITY & MONTE CARLO SIMULATION

Date: 10 April 2026
Scope: Risk parity optimizer, GARCH(1,1)-*t* volatility models, Cholesky-correlated Monte Carlo simulation, covariance estimation, supporting time-series pipeline
Codebase: `backend/calculator.py`, `backend/simulator.py`, `backend/main.py`, `backend/models.py`
Lines reviewed: ~530 (all quantitative logic)

> **ONE-LINE VERDICT**
> The core risk parity formulation is correct and the GARCH-*t* simulation architecture is well-chosen, but the dependence structure used in Monte Carlo path generation systematically understates joint tail risk, and the covariance estimator is inappropriate for the sample sizes and asset counts in play.

Page 1 of 18

Executive Verdict

PART A — TECHNICAL TRACK

The implementation makes defensible choices in its point estimates: GARCH(1,1) with Student-*t* innovations is an appropriate conditional heteroskedasticity model, the risk parity objective (minimise squared deviations of risk contributions from $1/N$) is standard, and SLSQP with inverse-vol initialisation is a reasonable solver. However, two structural errors undermine the simulation's distributional outputs. First, the use of a Gaussian copula with *t*-marginals (independent *t*-draws correlated via Cholesky) produces asymptotically independent tails, systematically understating the probability of simultaneous extreme losses —precisely the region VaR and CVaR are meant to measure. Second, the sample covariance matrix is estimated without shrinkage from ~50 effective observations for 14 assets, a regime where the Marchenko-Pastur bound guarantees that eigenvalues are noise-dominated, making the risk parity weights functions of estimation error as much as true risk structure.

PART B — PLAIN-TECHNICAL TRACK

The tool gets the big picture right: it uses a well-known volatility model (GARCH) with fat-tailed noise, and the risk parity math is correct. But the way it wires together the simulation has a flaw that matters: when it generates thousands of possible future paths, it underestimates the chance that *all* your stocks crash at the same time. The model gets everyday wiggles right but is too optimistic about worst-case scenarios. Additionally, when you hold 14 stocks across three different exchanges and only have ~50 usable days of data, the correlation estimates are noisy enough that the optimizer is partly chasing statistical ghosts rather than real portfolio structure.

2 CRITICAL **4** MATERIAL **4** MINOR

Page 2 of 18

Critical Findings

Page 3 of 18

Finding C1 — Gaussian copula understates joint tail dependence

DEPENDENCE STRUCTURE

PART A — TECHNICAL

`simulator.py:161-168`

The simulation generates innovations by drawing independent Student-*t* variates per asset and correlating them via a Cholesky factor of the standardised-residual correlation matrix:

```
for i in range(n_assets):
    raw_t = rng.standard_t(df=nu_i, size=num_simulations)
    innovations[:, i] = raw_t / np.sqrt(nu_i / (nu_i - 2))
correlated = innovations @ cholesky_L.T
```

This constructs a **meta-Gaussian (normal) copula with Student-*t* marginals**. The resulting joint distribution has the correct pairwise rank correlations and fat-tailed marginals, but its *tail dependence coefficient* is exactly zero:

$$\lambda_U = \lambda_L = lim_{u \to 1} P(X_2 > F_2^{-1}(u) \mid X_1 > F_1^{-1}(u)) = 0$$

By contrast, a multivariate Student-*t* distribution with ν degrees of freedom and correlation ρ has positive lower tail dependence:

$$\lambda_L = 2 \cdot t_{\nu+1}(-\sqrt{((\nu+1)(1-\rho)/(1+\rho))}) > 0$$

For typical equity parameters ($\nu \approx 5$, $\rho \approx 0.5$), this gives $\lambda_L \approx 0.15$–0.25. The Gaussian copula assigns probability ~0 to the event "all assets simultaneously in their 1% tail," while the multivariate-*t* assigns meaningful probability. Since VaR and CVaR are functions of exactly this tail region, they are systematically underestimated.

Literature: Embrechts, McNeil & Straumann (2002), "Correlation and Dependence in Risk Management: Properties and Pitfalls"; Demarta & McNeil (2005), "The *t* Copula and Related Copulas."

PART B — PLAIN-TECHNICAL

`simulator.py:161-168`

The simulation gives each stock its own fat-tailed random draw (Student-*t*), then uses a correlation matrix to make them move together. This gets normal-day correlations right, but it misses a crucial property of real markets: **correlations spike during crashes.**

Think of it this way: if NVDA drops 8% on a normal day, LRCX might drop 3%. The model handles that. But on a genuine crash day—a liquidity crisis, a macro shock—NVDA drops 8% and LRCX drops 7%, and AMAT drops 6%, and ASML drops 9%. The probability of *everything falling together* in the extreme tail is much higher in real life than this model allows.

The fix is to use a "multivariate Student-*t*" distribution, where all the stocks share a common shock factor. When that shared factor is extreme, *every* stock gets an extreme draw. This is a ~10-line code change (multiply

Page 4 of 18

all innovations by a common chi-squared scaling variable) and is the single most impactful improvement available.

> **Fix:** Replace independent *t*-draws + Cholesky with a proper multivariate Student-*t*: draw $Z \sim N(0, \Sigma)$ via Cholesky, draw $s \sim \chi^2(\nu)/\nu$, return $Z / \sqrt{s}$. Use the minimum fitted ν across assets (conservative) or a jointly estimated ν. This preserves all existing GARCH variance dynamics while correctly modelling tail dependence.

Page 5 of 18

Finding C2 — Covariance matrix estimated without shrinkage in high-dimension/low-sample regime

ESTIMATION

PART A — TECHNICAL

`calculator.py:75, 196`

The covariance matrix is estimated via the pandas sample estimator (`log_returns.cov()`) from T aligned observations of N assets. After the `dropna()` at line 75 removes all rows where any ticker has a NaN (cross-exchange calendar gaps), the effective T for a 14-asset, 3-exchange portfolio with a 63-day lookback is typically 48–55.

The concentration ratio is $N/T \approx 14/50 = 0.28$. By the Marchenko-Pastur law, in this regime the sample eigenvalue distribution is significantly distorted: the largest eigenvalue is inflated by a factor of approximately $(1 + \sqrt{(N/T)})^2 \approx 2.05$, and the smallest eigenvalues are compressed toward zero. The condition number is elevated not because of true portfolio structure but because of estimation noise.

The existing ridge regularisation (`calculator.py:133-136`) only triggers at condition number > 10^{10}, which addresses numerical singularity but does nothing for eigenvalue inflation at condition numbers of 10^2–10^4. The risk parity optimizer then allocates based on these noisy eigenvalues, potentially over-concentrating in assets whose pairwise correlations were *underestimated by chance* and under-weighting assets whose correlations were *overestimated by chance*.

Literature: Ledoit & Wolf (2004), "A Well-Conditioned Estimator for Large-Dimensional Covariance Matrices"; Ledoit & Wolf (2012), "Nonlinear Shrinkage Estimation of Large-Dimensional Covariance Matrices."

PART B — PLAIN-TECHNICAL

`calculator.py:75, 196`

To figure out how your 14 stocks move together, the code computes a 14×14 table of correlations and volatilities (the "covariance matrix"). That table has 105 unique entries. It's estimated from roughly 50 days of data—fewer than the entries it's trying to estimate.

When you estimate 105 things from 50 observations, some of the estimates will be too high and some too low, just by chance. The optimizer then treats those noisy estimates as truth and builds weights around them. The result: it might overweight a stock that *looked* uncorrelated with the rest in a small sample, when in reality it's quite correlated.

The standard fix is **shrinkage**—blending the raw sample estimate with a simpler, more stable estimate (like "all pairwise correlations are the same"). The Ledoit-Wolf estimator does this optimally and is available as a one-liner in scikit-learn. It would meaningfully stabilise the weights without changing the output when you have plenty of data.

> **Fix:** Replace `log_returns.cov().values * 252` with `LedoitWolf().fit(log_returns.values).covariance_ * 252` from `sklearn.covariance`.

> This is a one-line change that is strictly superior to the sample estimator in this regime and asymptotically equivalent when T is large.

Page 7 of 18

Material Findings

Finding M1 — Cross-exchange calendar gaps silently reduce effective sample size

DATA PIPELINE

PART A — TECHNICAL

`calculator.py:71-75`

```
log_returns = np.log(prices / prices.shift(1)).dropna(how="all")
log_returns = log_returns.tail(lookback_days)
log_returns = log_returns.dropna()  # drops row if ANY ticker has NaN
```

The `dropna()` call uses the default `how="any"`, discarding every row where at least one asset has no trade. For a portfolio spanning NYSE/NASDAQ (US), Euronext Paris (France), and KRX (Korea), the non-overlapping holiday calendars produce 15–25 mismatched days per year. With a 63-day lookback, this can reduce the effective sample to $T_{\text{eff}} \approx 45$–$52$.

Critically, the user-configured lookback is treated as the *maximum* window, but the covariance is estimated from $T_{\text{eff}} <$ lookback_days observations. This is not surfaced to the user, creating a false sense of estimation precision. It also interacts with Finding C2: the concentration ratio worsens from $N/T = 14/63 = 0.22$ to $14/48 = 0.29$.

PART B — PLAIN-TECHNICAL

`calculator.py:71-75`

When you set a 63-day lookback, you'd expect the tool to use 63 days of data. But since your portfolio includes stocks from the US, France, and Korea—which have different national holidays—any day where *any one* of those markets is closed gets thrown out entirely. You might end up with only 48 usable days, but nowhere does the tool tell you this.

This matters because the correlation estimates (Finding C2) are already stretched thin. Silently losing another 15–20% of your data makes them noisier, and the weights less reliable.

> **Fix:** (1) Return the effective observation count (`len(log_returns)`) in the API response so the frontend can display "Effective lookback: 48 of 63 days". (2) Consider pairwise covariance estimation instead of listwise deletion—compute each (i,j) pair using all dates where both *i* and *j* traded, then apply nearest-PSD projection. This maximises data use at a modest complexity cost.

Page 8 of 18

Finding M2 — No GARCH model adequacy diagnostics or residual checks

MODEL VALIDATION

PART A — TECHNICAL

`simulator.py:15-74`

The `_fit_single_garch` function fits GARCH(1,1)-*t* and extracts parameters without any residual diagnostics. Specifically:

- **No Ljung-Box test on standardised residuals.** If the GARCH(1,1) doesn't fully capture the autocorrelation structure of squared returns (e.g., because a higher order or an asymmetric specification like GJR-GARCH is needed), the residuals will exhibit serial dependence, violating the i.i.d. assumption used in the simulation.
- **No Kolmogorov-Smirnov or Anderson-Darling test of the *t*-distribution fit.** The innovation distribution is assumed Student-*t* but never verified. If the true distribution has, say, an asymmetric left tail, the simulation will under-generate extreme losses.
- **No sign-bias test.** The symmetric GARCH(1,1) treats positive and negative shocks identically. Equity returns exhibit well-documented leverage effects (Black, 1976): negative shocks increase future volatility more than positive shocks of equal magnitude. A GJR-GARCH or EGARCH would capture this.

The constant-variance fallback (lines 62–74) is triggered on *any* exception, including convergence warnings that might indicate a near-boundary solution rather than a failed fit. This conflates estimation failure with model misspecification.

PART B — PLAIN-TECHNICAL

`simulator.py:15-74`

The GARCH model is fit to each stock and its parameters are extracted, but the code never checks whether the model actually fits well. This is like fitting a line to data and using the slope without ever looking at the residual plot.

Three things go unchecked:

- **Are the leftovers random?** If the GARCH model isn't capturing all the volatility patterns, its "residuals" (leftover noise) will still show patterns. The simulation treats them as pure noise, which would make the generated paths unrealistically smooth or spiky in the wrong places.
- **Is the noise distribution right?** The code assumes the noise follows a Student-*t* distribution, but never tests this. If the real noise is more skewed (bigger left tail than right), the simulation will undercount extreme losses.
- **Do crashes and rallies affect vol differently?** In real markets, a 5% drop increases future volatility more than a 5% rally. The current GARCH(1,1) treats them symmetrically. A GJR-GARCH variant would capture this "leverage effect."

> **Fix:** After fitting, run `arch`'s built-in `result.diagnostic_summary()` or at minimum Ljung-Box on `result.std_resid**2` at lags [5, 10, 20]. Flag assets where the test rejects at 5%. Consider upgrading to GJR-

> GARCH(1,1) as the default specification—it nests GARCH(1,1) (the asymmetry parameter $\gamma = 0$) and is standard in equity vol modelling.

Finding M3 — GARCH forecast horizon exceeds model validity range

FORECAST VALIDITY

PART A — TECHNICAL

`simulator.py:124-188, models.py:39`

The GARCH(1,1) is fit from a lookback of typically 63 days (sometimes as few as 21) but used to simulate forward paths of up to 504 days (2 years). The multi-step-ahead conditional variance forecast of a GARCH(1,1) converges to the unconditional variance at rate $(\alpha + \beta)^h$. For typical equity parameters ($\alpha + \beta \approx 0.97$), the half-life is approximately $-\ln(2)/\ln(0.97) \approx 23$ days.

Beyond ~3 half-lives (~70 days), the GARCH conditional variance forecast is essentially the unconditional variance, and the simulation is effectively drawing from an i.i.d. *t*-distribution. The elaborate day-by-day GARCH recursion adds computational cost and code complexity for the portion of the path beyond ~70 days without adding meaningful conditional information. More importantly, regime changes, structural breaks, and parameter instability over a 2-year horizon make the fitted parameters unreliable—the simulation presents false precision.

PART B — PLAIN-TECHNICAL

`simulator.py:124-188`

GARCH models are good at short-term volatility forecasts: "volatility was high yesterday, so it'll probably be somewhat high today." But this memory fades. With typical stock parameters, the model's memory of "current" volatility levels has a half-life of about 23 trading days. By day 70 of a simulation, the model has essentially forgotten where it started and is just generating random noise from the long-run average.

When you simulate out to 1–2 years (252–504 days), the first month of paths reflects genuine GARCH dynamics, but the remaining 90%+ is effectively a random walk with fat tails. The model doesn't know about regime changes, recessions, or structural shifts over that horizon. The fan chart looks precise out to 2 years, but that precision is an illusion.

> **Fix:** This is partly inherent to the model class and partly a communication issue. (1) Display a note for horizons beyond 126 days that longer horizons increasingly reflect unconditional (average) vol rather than current conditions. (2) For ambitious users, consider a regime-switching extension (Markov-switching GARCH) that can model transitions between low-vol and high-vol regimes, giving more realistic long-horizon distributions.

Page 10 of 18

Finding M4 — Portfolio return aggregation uses log-return weighted average

PATH GENERATION

PART A — TECHNICAL

simulator.py:182-186

```
returns_decimal = returns_pct / 100.0
portfolio_return = returns_decimal @ weights
portfolio_values[:, day] = portfolio_values[:, day - 1] * np.exp(portfolio_return)
```

Here `returns_decimal` are log returns (the GARCH was fit on percentage log returns). The code computes $r_p = \Sigma\, w_i \cdot r_i$ (where $r_i = \log(P_{i,t}/P_{i,t-1})$) and then applies $\exp(r_p)$. But the log return of a portfolio is:

$$r_p = log(\Sigma\, w_i \cdot e^{r_i}) \neq \Sigma\, w_i \cdot r_i$$

By Jensen's inequality (exp is convex), $\exp(\Sigma\, w_i\, r_i) \leq \Sigma\, w_i \exp(r_i)$, so the code systematically understates the portfolio growth factor. The bias is a second-order term proportional to the cross-sectional variance of returns: approximately $\frac{1}{2}\, \Sigma\, w_i(r_i - \bar{r})^2$. For daily returns this is typically 1–5 basis points per day, but it compounds over 126+ days.

For a daily-rebalanced portfolio (which is the implicit assumption when weights are fixed), the correct update is:

$$V_t = V_{t-1} \cdot \Sigma\, w_i \cdot e^{r_i}$$

PART B — PLAIN-TECHNICAL

simulator.py:182-186

The code combines each stock's daily log return into a portfolio return by taking a weighted average, then converts back. This is mathematically slightly wrong—log returns don't combine across stocks by simple averaging (they only combine that way across *time*).

The error is small on any single day (a few basis points), but it always underestimates the portfolio's value and compounds over the simulation horizon. Over 126 days it might produce terminal values that are 0.5–2% too low—not catastrophic, but it introduces a pessimistic bias into every simulation run.

> **Fix:** Replace `portfolio_values[:, day] = portfolio_values[:, day - 1] * np.exp(portfolio_return)` with `portfolio_values[:, day] = portfolio_values[:, day - 1] * (np.exp(returns_decimal) @ weights)`. This computes the correct weighted average of gross returns.

Page 11 of 18

Minor Findings

Finding N1 — Annualised volatility statistic is inconsistent with GARCH dynamics

REPORTING

PART A — TECHNICAL

`simulator.py:233-235`

```
daily_equiv_vol = float(np.std(terminal_log_returns)) / np.sqrt(horizon_days)
annualized_vol = daily_equiv_vol * np.sqrt(252)
```

The $\sqrt{T}$ scaling assumes returns are i.i.d. Under GARCH, $\mathrm{Var}(r_1 + \ldots + r_T) \neq T \cdot \mathrm{Var}(r_1)$ because conditional variances are serially correlated. The reported "annualised vol" conflates two distinct quantities: (1) the cross-simulation dispersion of terminal outcomes, and (2) a time-scaled volatility measure. This number is not directly comparable to Bloomberg or other standard annualised vol figures.

PART B — PLAIN-TECHNICAL

`simulator.py:233-235`

The annualised vol number is computed using a shortcut ($\sqrt{252}$ scaling) that assumes each day's return is independent. But the whole point of GARCH is that days *aren't* independent—volatile days cluster together. This makes the reported vol a rough summary, not a precise risk measure. It won't match annualised vol figures from standard financial platforms.

> **Fix:** Either (a) compute annualised vol from the average of path-level realised variances: $\sqrt{(252/H \cdot \mathrm{mean}(\Sigma_t r_t^2))}$, or (b) label the statistic more precisely as "cross-simulation terminal dispersion" to avoid confusion.

Page 12 of 18

Finding N2 — VaR and CVaR computed at terminal date only

RISK METRICS

PART A — TECHNICAL

`simulator.py:237-241`

$VaR_{5\%}$ and $CVaR_{5\%}$ are computed from the distribution of terminal portfolio values $(P_T - P_0)/P_0$. These are *point-in-time* risk measures conditioned on holding to maturity. They do not capture the maximum loss experienced *during* the holding period. A path that drops 35% at day 40 and recovers to −5% at day 126 registers as a 5% loss in the VaR calculation, despite the holder experiencing a 35% drawdown.

The max drawdown statistic (lines 251–254) partially addresses this, but is reported as a median rather than a tail quantile. A "path-VaR" (5th percentile of the minimum portfolio value across each path) would be more informative for a holder who might need liquidity or face margin calls mid-horizon.

PART B — PLAIN-TECHNICAL

`simulator.py:237-241`

The VaR and CVaR numbers only look at where the portfolio ends up at the end of the simulation (e.g., day 126). But if your portfolio dropped 35% at day 40 before recovering to −5% at the end, the VaR says "−5%"—which misses the worst of the ride. The max drawdown stat is closer to what you'd actually experience, but it's reported as a median, not a worst-case measure.

> **Fix:** Add a "path VaR" metric: for each simulation, find the lowest portfolio value at any point, then report the 5th percentile of those minimums. This answers "what's the 95%-confidence worst dip I should be prepared for?"

Page 13 of 18

Finding N3 — Fixed RNG seed eliminates simulation variability

SIMULATION DESIGN

PART A — TECHNICAL

`simulator.py:138`

```
rng = np.random.default_rng(seed=42)
```

The fixed seed makes the simulation a deterministic function of (tickers, weights, lookback, horizon, num_simulations). While this is useful for reproducibility and testing, it means Monte Carlo error is not estimable from repeated runs, and users cannot assess whether 5,000 paths are sufficient. The 5th-percentile VaR from 5,000 paths has a standard error of approximately $\sqrt{(0.05 \cdot 0.95 / 5000)} / f(q_{0.05})$ where f is the terminal density at the quantile—potentially several percent of the VaR estimate itself.

PART B — PLAIN-TECHNICAL

`simulator.py:138`

Every time you click "Run Simulation" with the same inputs, you get the exact same results. This is because the random number generator uses a hardcoded starting point (seed=42). The simulation isn't really "running 5,000 scenarios"—it's computing one fixed answer. This is fine for a demo, but means clicking the button twice gives you no new information, and you can't tell whether 5,000 paths is "enough" for stable results.

> **Fix:** Use a random seed by default (`rng = np.random.default_rng()`) but accept an optional `seed` parameter for reproducibility. Alternatively, run two independent batches with different seeds and report the VaR difference as a confidence interval.

Page 14 of 18

Finding N4 — Correlation structure inconsistency between calculator and simulator

METHODOLOGY CONSISTENCY

PART A — TECHNICAL

`calculator.py:196` vs `simulator.py:94-121`

The risk parity optimizer computes weights using the sample covariance of raw log returns. The simulator generates correlated paths using the correlation matrix of GARCH standardised residuals. These are different objects: the raw covariance includes both the conditional mean variance and its time-variation, while the residual correlation strips out the GARCH-modelled heteroskedasticity.

In periods of non-stationary volatility (e.g., a vol spike during the lookback), the raw correlation can be inflated by common variance movements (a form of spurious correlation), while the residual correlation isolates the "structural" co-movement. The weights are optimised for one dependence structure but tested under another.

PART B — PLAIN-TECHNICAL

`calculator.py:196` vs `simulator.py:94-121`

The risk parity calculator and the Monte Carlo simulator use different methods to measure how stocks move together. The calculator uses raw return correlations; the simulator uses correlations after stripping out the GARCH volatility patterns. These can give different numbers, especially when volatility has been changing during the lookback period. The weights were optimised for one set of correlations but tested under a different set.

> **Fix:** This is a design choice, not a bug. Both approaches are defensible. If consistency is desired, use the GARCH residual correlation for both, or use the raw covariance for both. Document which is used where.

Page 15 of 18

What's Done Well

Strengths to preserve

1. RISK PARITY OBJECTIVE FORMULATION

`calculator.py:120-169` — The risk parity objective (minimise $\Sigma(RC_i - 1/N)^2$ subject to $\Sigma w = 1$, $w > 0$) is the standard Maillard, Roncalli & Teïletche (2010) formulation. The use of SLSQP with inverse-vol initialisation is a pragmatic choice—the objective landscape is well-behaved near the inverse-vol solution, and SLSQP handles the simplex constraint efficiently. The fallback to inverse-vol on optimizer failure is sensible since inverse-vol is itself a reasonable risk parity approximation when correlations are moderate.

2. GARCH(1,1)-*T* SPECIFICATION

`simulator.py:15-74` — The choice of GARCH(1,1) with Student-*t* innovations is the workhorse model of equity volatility for good reason: it captures volatility clustering, mean-reversion, and fat tails with only four parameters (ω, α, β, ν). The `mean="Zero"` specification is appropriate for daily returns where the drift is negligible relative to volatility. The stationarity enforcement ($\alpha + \beta < 1$) and variance capping (10× starting value) are sensible guardrails against parameter-space pathologies.

3. PARALLEL GARCH FITTING

`simulator.py:77-91` — Using `ThreadPoolExecutor` for per-asset GARCH fitting is a well-engineered decision. GARCH fitting is CPU-bound, and with 14 assets the parallelisation provides meaningful speedup. The code correctly handles the `arch` library's thread safety.

4. EIGENVALUE CLIPPING FOR PSD ENFORCEMENT

`simulator.py:112-118` — The residual correlation matrix is projected onto the positive-definite cone via eigenvalue clipping and re-normalisation before Cholesky decomposition. This is the Higham (2002) nearest-correlation-matrix approach and correctly prevents Cholesky failure from finite-sample negative eigenvalues.

5. BATCH DATA DOWNLOAD WITH ALIGNED RETURNS

`calculator.py:12-103` — Using a single `yf.download()` call for all tickers ensures calendar alignment. The error handling for missing tickers, insufficient data, and MultiIndex column formats is thorough and user-friendly. The 5-observation minimum is a reasonable floor for return computation.

6. VARIANCE EXPLOSION GUARDRAILS

`simulator.py:150-153, 175, 178-179` — The conditional variance cap at 10× starting value and daily return clamp at ±25% prevent the feedback loop where a large *t*-draw drives variance to infinity in subsequent steps. These are practical engineering choices that keep the simulation numerically stable without distorting the bulk of the distribution.

Page 16 of 18

Appendix (Technical Track Only)

A.1 — Tail Dependence Derivation

For a bivariate Gaussian copula with correlation ρ, the upper and lower tail dependence coefficients are both zero for $|\rho| < 1$:

$$\lambda_L = \lambda_U = 2 \cdot \lim_{x \to -\infty} \Phi(x\sqrt{((1-\rho)/(1+\rho))}) / \Phi(x) = 0$$

where Φ is the standard normal CDF. This holds regardless of the marginal distributions, because the copula governs the dependence structure. Hence, even with Student-*t* marginals (which have heavy tails individually), the probability of joint extreme events is asymptotically zero under a Gaussian copula.

For the multivariate Student-*t* copula with ν degrees of freedom:

$$\lambda_L = 2 \cdot t_{\nu+1}(-\sqrt{((\nu+1)(1-\rho)/(1+\rho))})$$

where $t_{\nu+1}$ is the CDF of the univariate Student-*t* with $\nu+1$ degrees of freedom. For the equity-typical values $\nu = 5$, $\rho = 0.5$:

$$\lambda_L = 2 \cdot t_6(-\sqrt{(6 \cdot 0.5/1.5)}) = 2 \cdot t_6(-1.414) \approx 2 \times 0.103 = 0.206$$

This means roughly 20% of the time that one asset is in its extreme left tail, the other is too. Under the Gaussian copula used in the current code, this probability converges to 0%.

A.2 — Marchenko-Pastur Bound on Eigenvalue Distortion

For an $N \times T$ random matrix with i.i.d. entries and $N/T \to y \in (0, 1]$, the sample eigenvalues of the covariance matrix are distributed over the interval:

$$[\sigma^2(1 - \sqrt{y})^2,\ \sigma^2(1 + \sqrt{y})^2]$$

For $N = 14$, $T = 50$, $y = 0.28$:

$$\lambda_{max} / \sigma^2 \approx (1 + \sqrt{0.28})^2 = (1.529)^2 = 2.34$$

The largest sample eigenvalue is inflated by ~134% relative to the true value, even when the true covariance is a scalar multiple of the identity. With structured covariance (as in equities), the distortion is somewhat less in the largest eigenvalue (market factor) but can be worse in smaller eigenvalues, leading to unstable eigenvector estimates that directly affect risk parity weights.

A.3 — Jensen's Inequality Bias in Log-Return Aggregation

Page 17 of 18

Let r_i denote the log return of asset i and w_i the portfolio weight. The code computes:

$$\hat{V}_t = V_{t-1} \cdot exp(\Sigma\, w_i\, r_i)$$

The correct daily-rebalanced portfolio value is:

$$V_t = V_{t-1} \cdot \Sigma\, w_i\, exp(r_i)$$

The bias per step is:

$$E[V_t - \hat{V}_t] / V_{t-1} \approx ½\, \Sigma\, w_i (r_i - \bar{r})^2$$

where $\bar{r} = \Sigma\, w_i\, r_i$ is the weighted average log return. This is the cross-sectional variance of log returns, which is second-order in daily return magnitudes. For typical daily equity moves ($\sigma \approx 1.5\%$), this is on the order of 0.5–5 basis points per day, compounding to 0.6–6.3% over 126 days. The bias always makes $\hat{V} < V$ (pessimistic).

A.4 — References

Bollerslev (1986)	"Generalized Autoregressive Conditional Heteroskedasticity." *Journal of Econometrics*, 31(3), 307–327.
Demarta & McNeil (2005)	"The *t* Copula and Related Copulas." *International Statistical Review*, 73(1), 111–129.
Embrechts, McNeil & Straumann (2002)	"Correlation and Dependence in Risk Management." In *Risk Management: Value at Risk and Beyond*, Cambridge UP.
Glosten, Jagannathan & Runkle (1993)	"On the Relation between the Expected Value and the Volatility of the Nominal Excess Return on Stocks." *Journal of Finance*, 48(5), 1779–1801.
Higham (2002)	"Computing the Nearest Correlation Matrix." *IMA Journal of Numerical Analysis*, 22(3), 329–343.
Ledoit & Wolf (2004)	"A Well-Conditioned Estimator for Large-Dimensional Covariance Matrices." *Journal of Multivariate Analysis*, 88(2), 365–411.
Maillard, Roncalli & Teïletche (2010)	"The Properties of Equally Weighted Risk Contribution Portfolios." *Journal of Portfolio Management*, 36(4), 60–70.
Marchenko & Pastur (1967)	"Distribution of Eigenvalues for Some Sets of Random Matrices." *Mathematics of the USSR-Sbornik*, 1(4), 457.